

0.86. L'alphabet en gras, avec poserS

ALPHABET_GRAS, un second alphabet de la console, aux traits épais. Le 0.10 avec ALPHABET_GRAS à la place d'ALPHABET : A à T sur la ligne 0, U à Z sur la ligne 1.

1. Ce qu'il faut comprendre

C'est le 0.10, avec un autre alphabet. Même boucle, même `poserS` : seul le nom du tableau change, `ALPHABET_GRAS` au lieu d'`ALPHABET`.

Ce qui est nouveau ici : `ALPHABET_GRAS`. Un second tableau de la console, comme `ALPHABET` : 26 cases, de `ALPHABET_GRAS[0]` (le A) à `ALPHABET_GRAS[25]` (le Z). Les lettres sont les mêmes, mais leurs traits font **2 pixels d'épaisseur** au lieu d'un.

L'ancien alphabet ne change pas : `ALPHABET` est toujours là, et tous les cours d'avant gardent leurs lettres. Les deux peuvent se mélanger dans un même programme.

`sizeof(ALPHABET_GRAS)` vaut 26, comme `sizeof(ALPHABET)` (le 0.9) : la boucle fait 26 tours, de A à Z.

`poserS` passe tout seul à la ligne suivante (le 0.10) : A à T sur la ligne 0, puis U à Z sur la ligne 1.

Gratuit si on ne s'en sert pas : les 26 lettres grasses ne sont ajoutées à la cartouche que si le programme écrit `ALPHABET_GRAS`.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <ALPHABET_GRAS>` → [0.85.1. ALPHABET GRAS — les lettres en gras](#)
- `#include <poserS>` → [0.9.3. La fonction poserS\(\) — poser, et passer à la ligne](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1394 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	78 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AdresseCase	10 o	← poser() — trouve l'adresse d'une case de l'écran
Diviser	17 o	← poser() — divise — le processeur ne sait pas le faire
Reste	10 o	← poser() — le reste d'une division
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	1120 o	← CopierTuiles — la police entière, et 26 tuiles dessinées
ALPHABET_GRAS_0	26 o	← poser() — une table gravée

Lettres gravées : toutes (`poserS()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser() × 1      53 o sur place, dans main() + AdresseCase (10 o), Diviser (17 o), Reste (10 o), ALPHABET_GRAS_0 (26 o)
image() × 1      3 o sur place, dans main() + AttendreImage (28 o)
poserS() × 1     0 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <ALPHABET_GRAS> // l'alphabet en gras : ALPHABET_GRAS[0] est le A gras
#include <poserS>         // pose une tuile, et passe à la ligne au bord
```

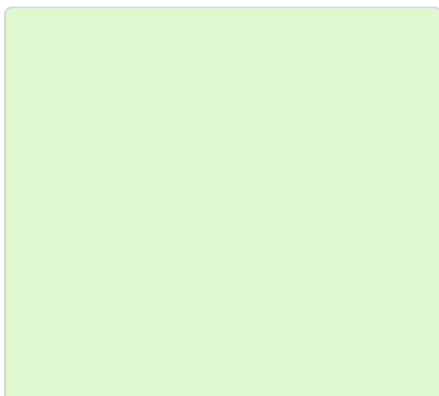
2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // La boucle, morceau par morceau :
    //
    //  poserS(i, 0, ALPHABET_GRAS[i]);
    //      | | |
    //      | | +-- ALPHABET_GRAS[i] : la lettre n° i, EN GRAS
    //      | |      (0 = A, 1 = B ... 25 = Z), comme ALPHABET[i]
    //      | +----- la ligne 0 ; poserS passe tout seul à la ligne 1
    //      +----- la colonne i : après la colonne 19, poserS repart en 0
    //
    //  i = 0 : A gras en (0, 0)      i = 19 : T gras en (19, 0)
    //  i = 20 : U gras en (0, 1)     i = 25 : Z gras en (5, 1)
    // sizeof(ALPHABET_GRAS) vaut 26 : i va de 0 (A) à 25 (Z).
    for (uint8_t i = 0; i < sizeof(ALPHABET_GRAS); i++) {
        poserS(i, 0, ALPHABET_GRAS[i]);
    }

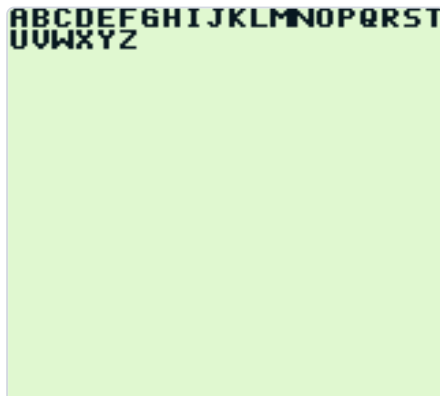
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Tout l'alphabet en gras, de A à Z : A à T sur la première ligne, U à Z sur la deuxième.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ 26 lettres grasses, toutes différentes (26)

✓ ce ne sont pas les lettres d'ALPHABET

Ensuite : **0.86.1. L'alphabet en gras, avec poserS — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++